ICPC 2026

PROBLEM	READ	SOLVED	WRITTEN	OPENED	COMMENT
A					
B					
C					
D					
E					
F					
G					
H					
I					
J					
K					
L					
M					
N					

Time	Andrey	Taisia	Timofey
0:30			
1:00			
1:30			
2:00			
2:30			
3:00			
3:30			
4:00			
4:30			
5:00			

Contents

1 Combinatorics

1.1 Gray Code

```
inline ll g(ll n) {
    return n ^ (n >> 1);
}
```

2 Convex Hull Trick

2.1 Convex Hull Trick

```
// monotone lines
struct ConvexHullTrick {
    // a, b of length k, p of lengths max(0, k - 1)
    // all lines is a[i] * x + b[i]
    // and line i > 0 maximum from x = p[i - 1] to x = p[i] - 1
    vector<ll> a, b, p;

    // a1 < a2
    static ll intersect(ll a1, ll b1, ll a2, ll b2) {
        ll den = a2 - a1;
        ll num = b1 - b2;
        if (num >= 0) {
            return (num + den - 1) / den;
        } else {
            return num / den;
        }
    }

    // na >= a[i] forall i
    void add_line(ll na, ll nb) {
        if (a.empty()) {
            a.push_back(na);
            b.push_back(nb);
            return;
        }
        if (a.back() > na) {
            return;
        }
        while (!p.empty() && a.back() * p.back() + b.back() <= na * p.
    back() + nb) {
            p.pop_back();
            a.pop_back();
            b.pop_back();
        }
        // else na == a.back() and in one concrete point
        // new line leq of old > in all points new line >
        if (a.back() < na) {
            p.push_back(intersect(a.back(), b.back(), na, nb));
            a.push_back(na);
            b.push_back(nb);
        }
    }

    ll query(ll x) {
        if (a.empty()) {
            return -inf;
        }
        ll l = 0, r = (ll)a.size();
        while (r - l > 1) {
            ll m = (r + l) / 2;
            if (p[m - 1] <= x) {
                l = m;
            } else {
                r = m;
            }
        }
        return a[l] * x + b[l];
    }
};
```

3 Convolutions

3.1 FFT MOD

```
const ll MOD = 998244353;
```

```
const ll ROOT = 31;
const ll REV_ROOT = rev(ROOT);
const int ROOT_PW = (1ll << 23);

vector<vector<ll>> w;

void precalc_ws(ll lg_n) {
    w.resize(lg_n + 1);
    w[0].resize(1, 1);
    for (int l = 1; l <= lg_n; ++l) {
        ll n = (1ll << l);
        w[l].resize(n / 2);

        ll fw = ROOT;
        for (int i = 0; i < 23 - l; ++i) {
            fw = (fw * fw) % MOD;
        }
        for (int j = 0; j < (n >> 1); ++j) {
            if (j % 2 == 0) {
                w[l][j] = w[l - 1][j / 2];
            } else {
                w[l][j] = (w[l - 1][j / 2] * fw) % MOD;
            }
        }
    }
}

void fft_inpl(vector<ll>& a, bool invert) {
    int n = a.size();

    // apply permutation by bit_rev
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2, lg = 1; len <= n; len <<= 1, ++lg) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < (len >> 1); ++j) {
                ll root = w[lg][j];
                if (invert) {
                    root = rev(root);
                }
                ll u = a[i + j], v = (a[i + j + len / 2] * root) % MOD;
                a[i + j] = u + v < MOD ? u + v : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
            }
        }
    }
    if (invert) {
        int nrev = rev(n);
        for (int i = 0; i < n; ++i) {
            a[i] *= nrev;
            a[i] %= MOD;
        }
    }
}

vector<ll> vect_mult(const vector<ll>& a, const vector<ll>& b) {
    ll sz = a.size() + b.size() - 1;
    ll n = 1;
    int lg_n = 0;
    while (n < sz) {
        n <<= 1;
        ++lg_n;
    }
    if (w.size() <= lg_n) {
        precalc_ws(lg_n);
    }

    vector<ll> fa(n, 0), fb(n, 0);
    for (int i = 0; i < n; ++i) {
        if (i < a.size()) {
            fa[i] = a[i];
        }
        if (i < b.size()) {
            fb[i] = b[i];
        }
    }

    fft_inpl(fa, false);
    fft_inpl(fb, false);
    for (int i = 0; i < n; ++i) {
        fa[i] *= fb[i];
```

```
        fa[i] %= MOD;
    }
    fft_inpl(fa, true);

    vector<ll> mult(sz);
    for (int i = 0; i < sz; ++i) {
        mult[i] = fa[i];
    }
    return mult;
}
```

3.2 FFT

```
vector<vector<comp>> w;

void precalc_ws(ll lg_n) {
    w.resize(lg_n + 1);
    w[0].resize(1, 1);
    for (int l = 1; l <= lg_n; ++l) {
        ll n = (1ll << l);
        w[l].resize(n / 2);

        // need cosl, sinl for complex<ld>
        comp fw(cos(2 * PI / n), sin(2 * PI / n));
        for (int j = 0; j < (n >> 1); ++j) {
            if (j % 2 == 0) {
                w[l][j] = w[l - 1][j / 2];
            } else {
                w[l][j] = w[l - 1][j / 2] * fw;
            }
        }
    }
}

void fft_inpl(vector<comp>& a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2, lg = 1; len <= n; len <<= 1, ++lg) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < (len >> 1); ++j) {
                comp root = w[lg][j];
                if (invert)
                    root = conj(root);
                comp u = a[i + j], v = a[i + j + (len >> 1)] * root;
                a[i + j] = u + v;
                a[i + j + (len >> 1)] = u - v;
            }
        }
    }
    if (invert) {
        double rev_n = 1.0 / n;
        for (int i = 0; i < n; ++i) {
            a[i] *= rev_n;
        }
    }
}

vector<ll> vect_mult(const vector<ll>& a, const vector<ll>& b) {
    ll sz = a.size() + b.size() - 1;
    ll n = 1;
    int lg_n = 0;
    while (n < sz) {
        n <<= 1;
        ++lg_n;
    }
    if (w.size() <= lg_n) {
        precalc_ws(lg_n);
    }

    vector<comp> p(n, 0);
    for (int i = 0; i < n; ++i) {
        p[i] = comp(i < a.size() ? a[i] : 0, i < b.size() ? b[i] : 0);
    }

    fft_inpl(p, false);
    for (int i = 0; i < n; ++i) {
        p[i] *= p[i];
    }
    fft_inpl(p, true);
```

```
    vector<ll> mult(sz);
    for (int i = 0; i < sz; ++i) {
        // need roundl for complex<ld>
        mult[i] = round(p[i].imag() * 0.5);
    }
    return mult;
}
```

3.3 FHT

```
inline ll soft_mod(ll a) {
    return a >= MOD ? a - MOD : a;
}

void FHT(vector<ll>& a) {
    ll sz = a.size();
    for (int mid = 1; mid < sz; mid <<= 1) {
        for (int i = 0; i < sz; i += mid << 1) {
            for (int j = 0; j < mid; ++j) {
                ll x = a[i + j];
                ll y = a[i + j + mid];
                a[i + j] = soft_mod(x + y);
                a[i + j + mid] = soft_mod(x - y + MOD);
            }
        }
    }
}

vector<ll> xor_mult(vector<ll> a, vector<ll> b) {
    FHT(a);
    FHT(b);
    ll n = a.size();
    vector<ll> c(n);

    for (int i = 0; i < n; ++i) {
        c[i] = (a[i] * b[i]) % MOD;
    }

    FHT(c);
    ll revn = bin_pow(n, MOD - 2);
    for (int i = 0; i < n; ++i) {
        c[i] *= revn;
        c[i] %= MOD;
    }
    return c;
}
```

3.4 SOS DP

```
void sos(vector<ll>& dp) {
    ll n = dp.size();
    ll k = 0;
    while ((1ll << k) < n) {
        ++k;
    }

    for (int j = 0; j < k; ++j) {
        for (int i = 0; i < n; ++i) {
            if (i & (1ll << j)) {
                dp[i] += dp[i ^ (1ll << j)];
            }
        }
    }
}

void revsos(vector<ll>& dp) {
    ll n = dp.size();

    ll k = 0;
    while ((1ll << k) < n) {
        ++k;
    }

    for (int j = k - 1; j >= 0; --j) {
        for (int i = 0; i < n; ++i) {
            if (i & (1ll << j)) {
                dp[i] -= dp[i ^ (1ll << j)];
            }
        }
    }
}
```

4 Data Structures

4.1 Centroid Decomposition

```
struct CentroidDecomp {
    ll n;
    vector<ll> parent;

    void calc_sizes(ll v, vector<char>& used, vector<vector<ll>>& g,
    vector<ll>& s) {
        s[v] = 1;
        used[v] = 1;

        for (ll u : g[v]) {
            if (!used[u]) {
                calc_sizes(u, used, g, s);
                s[v] += s[u];
            }
        }
    }

    ll find_centroid(ll v, vector<char>& used, vector<vector<ll>>& g,
    vector<ll>& s, ll sz) {
        used[v] = 1;

        for (ll u : g[v]) {
            if (!used[u]) {
                if (s[u] > sz / 2) {
                    return find_centroid(u, used, g, s, sz);
                }
            }
        }

        return v;
    }

    void set_parent(ll v, vector<char>& used, vector<vector<ll>>& g, ll
    cent) {
        if (v != cent)
            parent[v] = cent;
        used[v] = 1;

        for (ll u : g[v]) {
            if (!used[u]) {
                set_parent(u, used, g, cent);
            }
        }
    }

    CentroidDecomp(ll N, vector<vector<ll>>& g) {
        n = N;
        parent.assign(n, -1);

        vector<char> cused(n, 0);
        vector<ll> s(n, 0);
        ll cnt_used = 0;

        while (cnt_used < n) {
            vector<char> sused = cused, used = cused, pused = cused;

            for (int i = 0; i < n; ++i) {
                if (!sused[i]) {
                    calc_sizes(i, sused, g, s);
                    ll cent = find_centroid(i, used, g, s, s[i]);

                    cused[cent] = 1;
                    ++cnt_used;
                    set_parent(i, pused, g, cent);
                }
            }
        }
    }
};
```

4.2 DSU

```
struct DSU {
    ll n;
    vector<ll> prev, size;

    DSU(ll N) : n(N) {
        prev.assign(n, -1);
        size.assign(n, 1);
    }

    ll root(ll a) {
        ll aa = a;
        while (prev[a] != -1)
            a = prev[a];

        while (prev[aa] != -1) {
```

```
            ll last = prev[aa];
            prev[aa] = a;
            aa = last;
        }
        return a;
    }

    bool connected(ll a, ll b) {
        return root(a) == root(b);
    }

    bool unite(ll a, ll b) {
        a = root(a);
        b = root(b);
        if (a == b)
            return false;
        if (size[a] < size[b])
            swap(a, b);
        prev[b] = a;
        size[a] += size[b];
        return true;
    }
};
```

4.3 Disjoint Sparse Table

```
struct DisjointSparseTable {
    vector<vector<ll>> dp;

    ll neutral() {
        return 0;
    }
    ll merge(ll a, ll b) {
        return a + b;
    }

    DisjointSparseTable() {
    }

    DisjointSparseTable(const vector<ll>& a) {
        ll n = a.size();
        ll k = 0;
        ll nn = 1;
        while (nn <= n) {
            ++k;
            nn <<= 1;
        }

        dp.assign(k, vector<ll>(n + 1, neutral()));

        ll t = 0;
        for (ll len = 1; len <= n; len <<= 1, ++t) {
            for (ll cent = len; cent <= n; cent += len * 2) {
                dp[t][cent - 1] = a[cent - 1];
                for (ll i = cent - 2; i >= cent - len; --i) {
                    dp[t][i] = merge(a[i], dp[t][i + 1]);
                }

                dp[t][cent] = neutral();
                for (ll i = cent + 1; i < min(cent + len, n + 1); ++i) {
                    dp[t][i] = merge(dp[t][i - 1], a[i - 1]);
                }
            }
        }
    }

    // query on [l, r)
    ll query(ll l, ll r) {
        if (r <= l)
            return neutral();
        ll t = 63 - __builtin_clzll((ull)(l ^ r));
        return merge(dp[t][l], dp[t][r]);
    }
};
```

4.4 Fenwick 2D

```
template <class T>
struct Fenwick2D {
    ll n = 0, m = 0;
    vector<vector<T>> bit;

    Fenwick2D() = default;
    Fenwick2D(int n_, int m_) {
        init(n_, m_);
    }

    void init(int n_, int m_) {
```

```
        n = n_;
        m = m_;
        bit.assign((int)n + 1, vector<T>((int)m + 1, T{}));
    }

    // add delta at (x, y), 0-indexed
    void add(ll x, ll y, T delta) {
        for (ll i = x + 1; i <= n; i += i & -i)
            for (ll j = y + 1; j <= m; j += j & -j)
                bit[i][j] += delta;
    }

    // sum over [0..x] x [0..y], 0-indexed; returns 0 if x<0 or y<0
    T sum_prefix(ll x, ll y) const {
        if (x < 0 || y < 0)
            return T{};
        T res{};
        for (ll i = x + 1; i > 0; i -= i & -i)
            for (ll j = y + 1; j > 0; j -= j & -j)
                res += bit[i][j];
        return res;
    }

    // sum over rectangle [x1..x2] x [y1..y2], inclusive, 0-indexed
    T sum_rect(ll x1, ll y1, ll x2, ll y2) const {
        return sum_prefix(x2, y2) - sum_prefix(x1 - 1, y2) - sum_prefix(
    x2, y1 - 1) + sum_prefix(x1 - 1, y1 - 1);
    }
};
```

4.5 Fenwick Tree

```
struct FenwickTree {
    vector<ll> tree;
    ll n;

    FenwickTree(ll N) {
        n = N;
        tree.assign(n, 0);
    }

    ll F(ll x) {
        return x & (x + 1);
    }

    ll H(ll x) {
        return x | (x + 1);
    }

    // [0, r)
    ll query(ll r) {
        if (r > n) {
            r = n;
        }
        --r;
        ll ans = 0;
        while (r >= 0) {
            ans ^= tree[r];
            r = F(r) - 1;
        }
        return ans;
    }

    // [l, r)
    ll query(ll l, ll r) {
        return query(r) ^ query(l);
    }

    void add(ll id, ll val) {
        while (id < n) {
            tree[id] ^= val;
            id = H(id);
        }
    }
};
```

4.6 HLD

```
struct HLD {
    ll n;
    vector<ll> top, pos, h, par;

    ll gpos = 0;

    SegmentTree st;

    void calc_sizes(ll v, vector<char>& used, const vector<vector<ll>>& g
    , vector<ll>& s) {
```

```
    s[v] = 1;
    used[v] = 1;

    for (ll u : g[v]) {
        if (!used[u]) {
            calc_sizes(u, used, g, s);
            s[v] += s[u];
        }
    }
}

void build_dec(ll v, ll p, ll pr, vector<char>& used, const vector<
vector<ll>>& g, vector<ll>& s) {
    used[v] = 1;
    pos[v] = gpos++;
    top[pos[v]] = pos[p];
    par[pos[v]] = pos[pr];
    if (pr != v)
        h[pos[v]] = h[pos[pr]] + 1;

    vector<ll> child;

    for (ll u : g[v]) {
        if (!used[u]) {
            child.push_back(u);
        }
    }

    for (int i = 1; i < child.size(); ++i) {
        if (s[child[i]] > s[child[0]]) {
            swap(child[i], child[0]);
        }
    }

    if (!child.empty()) {
        build_dec(child[0], p, v, used, g, s);
        for (int i = 1; i < child.size(); ++i) {
            build_dec(child[i], child[i], v, used, g, s);
        }
    }
}

HLD(ll N, const vector<vector<ll>>& g, const vector<ll>& a) {
    n = N;
    top.resize(n, 0);
    pos.resize(n, 0);
    h.resize(n, 0);
    par.resize(n, 0);
    vector<char> used(n, 0);
    vector<ll> s(n, 0);

    calc_sizes(0, used, g, s);
    used.assign(n, 0);
    build_dec(0, 0, 0, used, g, s);

    vector<ll> pa(n, 0);
    for (int i = 0; i < n; ++i) {
        pa[pos[i]] = a[i];
    }

    st = SegmentTree(pa);
}

ll query(ll u, ll v) {
    ll ans = 0;

    u = pos[u];
    v = pos[v];

    ll pu = top[u];
    ll pv = top[v];

    while (pu != pv) {
        if (h[pu] >= h[pv]) {
            ans += st.query(pu, u + 1).sum;
            u = par[pu];
            pu = top[u];
        } else {
            ans += st.query(pv, v + 1).sum;
            v = par[pv];
            pv = top[v];
        }
    }
    ans += st.query(min(u, v), max(u, v) + 1).sum;

    return ans;
}
```

```
    void change(ll u, ll val) {
        u = pos[u];
        st.change(u, val);
    }
};
```

4.7 Implicit Segment Tree

```
struct ImplicitSegmentTree {
    struct Node {
        Node* left = nullptr;
        Node* right = nullptr;
        ll sum = 0;
        ll add = 0;
    };

    Node neutral = Node();
    ll n;
    Node* root = nullptr;

    Node merge(Node n1, Node n2) {
        Node res;
        res.sum = n1.sum + n2.sum;
        return res;
    }

    void fix(Node* node) {
        node->sum = 0;
        if (node->left) {
            node->sum += node->left->sum;
        }
        if (node->right) {
            node->sum += node->right->sum;
        }
    }

    void apply(Node* node, ll l, ll r, ll val) {
        node->sum += val * (r - l);
        node->add += val;
    }

    void push(Node* node, ll l, ll r) {
        if (node->add == 0) {
            return;
        }
        ll m = (l + r) / 2;
        if (!node->left) {
            node->left = new Node();
        }
        if (!node->right) {
            node->right = new Node();
        }
        apply(node->left, l, m, node->add);
        apply(node->right, m, r, node->add);
        node->add = 0;
    }

    ImplicitSegmentTree() {
    }

    ImplicitSegmentTree(ll N) {
        n = N;
    }

    ImplicitSegmentTree(const vector<ll>& start) {
        n = start.size();
        for (ll i = 0; i < n; i++) {
            change(i, start[i]);
        }
    }

    void change(Node*& node, ll l, ll r, ll id, ll val) {
        if (!node) {
            node = new Node();
        }
        if (l + 1 == r) {
            node->sum = val;
            node->add = 0;
            return;
        }
        push(node, l, r);
        ll m = (l + r) / 2;
        if (id < m) {
            change(node->left, l, m, id, val);
        } else {
            change(node->right, m, r, id, val);
        }
```

```
    fix(node);
}

void change(ll id, ll val) {
    change(root, 0, n, id, val);
}

void add(Node*& node, ll l, ll r, ll ql, ll qr, ll val) {
    if (qr <= l || r <= ql) {
        return;
    }
    if (!node) {
        node = new Node();
    }
    if (ql <= l && r <= qr) {
        apply(node, l, r, val);
        return;
    }
    push(node, l, r);
    ll m = (l + r) / 2;
    add(node->left, l, m, ql, qr, val);
    add(node->right, m, r, ql, qr, val);
    fix(node);
}

void add(ll ql, ll qr, ll val) {
    add(root, 0, n, ql, qr, val);
}

Node query(Node* node, ll l, ll r, ll ql, ll qr) {
    if (!node || qr <= l || r <= ql) {
        return neutral;
    }
    if (ql <= l && r <= qr) {
        return *node;
    }
    push(node, l, r);
    ll m = (l + r) / 2;
    return merge(query(node->left, l, m, ql, qr), query(node->right,
m, r, ql, qr));
}

Node query(ll ql, ll qr) {
    return query(root, 0, n, ql, qr);
}
};
```

4.8 Persistent DSU

```
struct PersistentDSU {
    ll n;
    vector<ll> prev, size;

    PersistentDSU(ll N) : n(N) {
        prev.assign(n, -1);
        size.assign(n, 1);
    }

    ll root(ll a) {
        while (prev[a] != -1)
            a = prev[a];
        return a;
    }

    bool connected(ll a, ll b) {
        return root(a) == root(b);
    }

    vector<pair<ll, ll>> changes;  //

    bool unite(ll a, ll b) {
        a = root(a);
        b = root(b);
        if (a == b)
            return false;
        if (size[a] < size[b])
            swap(a, b);
        prev[b] = a;
        size[a] += size[b];
        changes.push_back({b, a});
        return true;
    }

    bool rollback() {
        if (changes.empty())
            return false;
        pair<ll, ll> ba = changes.back();
        changes.pop_back();
```

```
        prev[ba.first] = -1;
        size[ba.second] -= size[ba.first];
        return true;
    }
};
```

4.9 Prefix Automate

```
struct PrefixAutomate {
    const static int A = 26;

    struct Node {
        int next[A] = {-1};
        int link = -1;

        int prev = -1;
        int pe = -1;

        int has = 0;
    };

    vector<Node> tree;

    PrefixAutomate() {
        tree.push_back(Node());
    }

    void add(const string& s) {
        int v = 0;
        for (char c : s) {
            int e = c - 'a';
            if (tree[v].next[e] == -1) {
                tree[v].next[e] = tree.size();
                tree.push_back(Node());
                tree.back().prev = v;
                tree.back().pe = e;
            }
            v = tree[v].next[e];
        }
        tree[v].has = 1;
    }

    void addVect(const vector<string>& vs) {
        for (const auto& s : vs) {
            add(s);
        }
    }
}

void del(const string& s) {
    int v = 0;
    for (char c : s) {
        int e = c - 'a';
        if (tree[v].next[e] == -1) {
            return;
        }
        v = tree[v].next[e];
    }
    tree[v].has = 0;
}

void ahoCorasick() {
    queue<int> que;

    tree[0].prev = 0;
    tree[0].pe = -1;

    tree[0].link = 0;
    for (int i = 0; i < A; ++i) {
        if (tree[0].next[i] == -1) {
            tree[0].next[i] = 0;
        } else {
            que.push(tree[0].next[i]);
        }
    }

    while (!que.empty()) {
        int v = que.front();
        que.pop();

        if (tree[v].link == -1) {
            if (v == 0 || tree[v].prev == 0)
                tree[v].link = 0;
            else
                tree[v].link = tree[tree[v].prev].next[tree[v].pe];
        }

        for (int i = 0; i < A; ++i) {
            if (tree[v].next[i] == -1) {
```

```
                    tree[v].next[i] = tree[tree[v].link].next[i];
                } else {
                    que.push(tree[v].next[i]);
                }
            }
        }
    }
};
```

4.10 Segment Tree Beats

```
struct SegmentTreeBeats {
    struct Node {
        ll sum = 0;
        ll firstMax = -inf;
        ll cntMax = 0;
        ll secondMax = -inf;

        ll min_eq = inf;

        Node(ll Sum = 0, ll FirstMax = -inf, ll CntMax = 0, ll SecondMax
    = -inf) {
            sum = Sum;
            firstMax = FirstMax;
            cntMax = CntMax;
            secondMax = SecondMax;
        }
    };

    Node fromOne(ll a) {
        return Node(a, a, 1, -inf);
    }

    Node neutral = Node();
    ll n;
    vector<Node> tree;

    SegmentTreeBeats(const vector<ll>& start) {
        n = start.size();
        tree.resize(4 * n + 3);
        init(start);
    }

    SegmentTreeBeats(ll N) {
        n = N;
        tree.resize(4 * n + 3);
        vector<ll> start(n, 0);
        init(start);
    }

    Node merge(Node n1, Node n2) {
        Node res(n1.sum + n2.sum);

        if (n1.firstMax > n2.firstMax) {
            res.firstMax = n1.firstMax;
            res.cntMax = n1.cntMax;
            res.secondMax = max(n1.secondMax, n2.firstMax);
        } else if (n1.firstMax < n2.firstMax) {
            res.firstMax = n2.firstMax;
            res.cntMax = n2.cntMax;
            res.secondMax = max(n1.firstMax, n2.secondMax);
        } else {
            res.firstMax = n1.firstMax;
            res.cntMax = n1.cntMax + n2.cntMax;
            res.secondMax = max(n1.secondMax, n2.secondMax);
        }
        return res;
    }

    void fix(ll v, ll l, ll r) {
        tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
    }

    void init(ll v, ll l, ll r, const vector<ll>& start) {
        if (l + 1 == r) {
            tree[v] = fromOne(start[l]);
            return;
        }

        ll m = (r + l) / 2;
        init(v * 2 + 1, l, m, start);
        init(v * 2 + 2, m, r, start);
        fix(v, l, r);
    }

    void init(const vector<ll>& start) {
        init(0, 0, n, start);
    }
```

```
    // [l: r)
    Node query(ll v, ll l, ll r, ll ql, ll qr) {
        if (ql <= l && r <= qr) {
            return tree[v];
        }
        if (r <= ql || qr <= l) {
            return neutral;
        }

        ll m = (r + l) / 2;
        push(v, l, r);
        return merge(query(v * 2 + 1, l, m, ql, qr), query(v * 2 + 2, m,
    r, ql, qr));
    }

    Node query(ll ql, ll qr) {
        return query(0, 0, n, ql, qr);
    }

    bool breakCondition(ll v, ll l, ll r, ll val) {
        return tree[v].firstMax <= val;
    }

    bool tagCondition(ll v, ll l, ll r, ll val) {
        return tree[v].secondMax < val;
    }

    void apply(ll v, ll l, ll r, ll val) {
        // only if tagCondition() == true
        ll diff = min(tree[v].firstMax, val) - tree[v].firstMax;
        tree[v].sum += tree[v].cntMax * diff;
        tree[v].firstMax += diff;

        tree[v].min_eq = min(val, tree[v].min_eq);
    }

    void push(ll v, ll l, ll r) {
        ll m = (r + l) / 2;

        apply(v * 2 + 1, l, m, tree[v].min_eq);
        apply(v * 2 + 2, m, r, tree[v].min_eq);
        tree[v].min_eq = inf;
    }

    void update(ll v, ll l, ll r, ll ql, ll qr, ll val) {
        if (r <= ql || qr <= l || breakCondition(v, l, r, val)) {
            return;
        }
        if (ql <= l && r <= qr && tagCondition(v, l, r, val)) {
            apply(v, l, r, val);
            return;
        }

        ll m = (r + l) / 2;
        push(v, l, r);
        update(v * 2 + 1, l, m, ql, qr, val);
        update(v * 2 + 2, m, r, ql, qr, val);
        fix(v, l, r);
    }

    void update(ll ql, ll qr, ll val) {
        update(0, 0, n, ql, qr, val);
    }
};
```

4.11 Segment Tree Mass

```
struct SegmentTreeMass {
    struct Node {
        ll sum = 0;
        ll add = 0;

        Node(ll Sum = 0, ll Add = 0) {
            sum = Sum;
            add = Add;
        }
    };

    Node neutral = Node();
    ll n;
    vector<Node> tree;

    SegmentTreeMass(const vector<ll>& start) {
        n = start.size();
        tree.resize(4 * n + 3);
        init(start);
    }
```

```
    SegmentTreeMass(ll N) {
        n = N;
        tree.resize(4 * n + 3);
        vector<ll> start(n, 0);
        init(start);
    }

    Node merge(Node n1, Node n2) {
        return Node(n1.sum + n2.sum);
    }

    void fix(ll v, ll l, ll r) {
        tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
    }

    void apply(ll v, ll l, ll r, ll val) {
        tree[v].add += val;
        tree[v].sum += val * (r - l);
    }

    void push(ll v, ll l, ll r) {
        ll m = (r + l) / 2;

        apply(v * 2 + 1, l, m, tree[v].add);
        apply(v * 2 + 2, m, r, tree[v].add);
        tree[v].add = 0;
    }

    void init(ll v, ll l, ll r, const vector<ll>& start) {
        if (l + 1 == r) {
            tree[v] = Node(start[l], 0);
            return;
        }

        ll m = (r + l) / 2;
        init(v * 2 + 1, l, m, start);
        init(v * 2 + 2, m, r, start);
        fix(v, l, r);
    }

    void init(const vector<ll>& start) {
        init(0, 0, n, start);
    }

    // [l: r)
    Node query(ll v, ll l, ll r, ll ql, ll qr) {
        if (ql <= l && r <= qr) {
            return tree[v];
        }
        if (r <= ql || qr <= l) {
            return neutral;
        }

        ll m = (r + l) / 2;
        push(v, l, r);
        return merge(query(v * 2 + 1, l, m, ql, qr), query(v * 2 + 2, m,
    r, ql, qr));
    }

    Node query(ll ql, ll qr) {
        return query(0, 0, n, ql, qr);
    }

    void add(ll v, ll l, ll r, ll ql, ll qr, ll val) {
        if (ql <= l && r <= qr) {
            apply(v, l, r, val);
            return;
        }
        if (r <= ql || qr <= l) {
            return;
        }

        ll m = (r + l) / 2;
        push(v, l, r);
        add(v * 2 + 1, l, m, ql, qr, val);
        add(v * 2 + 2, m, r, ql, qr, val);
        fix(v, l, r);
    }

    void add(ll ql, ll qr, ll val) {
        add(0, 0, n, ql, qr, val);
    }
};
```

4.12 Segment Tree

```
struct SegmentTree {
```

```
struct Node {
    ll sum;

    Node(ll Sum = 0) : sum(Sum) {
    }
};

Node neutral = Node();

ll n;
vector<Node> tree;

void init(ll v, ll l, ll r, const vector<ll>& start) {
    if (l + 1 == r) {
        tree[v] = Node(start[l]);
        return;
    }

    ll m = (l + r) / 2;

    init(v * 2 + 1, l, m, start);
    init(v * 2 + 2, m, r, start);
    tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
}

SegmentTree() {
}

SegmentTree(const vector<ll>& start) {
    n = start.size();
    tree.resize(n * 4 + 3);
    init(0, 0, n, start);
}

Node merge(Node n1, Node n2) {
    return Node(n1.sum + n2.sum);
}

Node query(ll v, ll l, ll r, ll ql, ll qr) {
    if (ql <= l && r <= qr)
        return tree[v];
    if (qr <= l || r <= ql)
        return neutral;
    ll m = (l + r) / 2;
    return merge(query(v * 2 + 1, l, m, ql, qr), query(v * 2 + 2, m,
r, ql, qr));
    }

    Node query(ll ql, ll qr) {
        return query(0, 0, n, ql, qr);
    }

    void change(ll v, ll l, ll r, ll id, ll val) {
        if (l + 1 == r && l == id) {
            tree[v] = Node(val);
            return;
        }
        if (r <= id || id < l) {
            return;
        }

        ll m = (l + r) / 2;
        change(v * 2 + 1, l, m, id, val);
        change(v * 2 + 2, m, r, id, val);
        tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
    }

    void change(ll id, ll val) {
        change(0, 0, n, id, val);
    }
};
```

4.13 Sparse Table

```
template <class T>
struct SparseTable {
    ll n, k;
    vector<T> a;
    vector<vector<T>> sparse;
    vector<ll> length;

    SparseTable(vector<T>& start) {
        n = start.size();
        k = 0;
        for (ll d = 1; d <= n; d *= 2, ++k) {
        }
        a.assign(start.begin(), start.end());
```

```
        sparse.assign(n, vector<T>(k));
        countSparse();
        precalc();
    }

    void precalc() {
        length.assign(n + 1, 0);
        ll t = 0;
        for (int l = 1; l <= n; ++l) {
            if ((1ll << (t + 1)) < l)
                ++t;
            length[l] = t;
        }
    }

    void countSparse() {
        for (int i = 0; i < n; ++i) {
            sparse[i][0] = a[i];
        }
        for (int j = 1; j < k; ++j) {
            for (int i = 0; i < n; ++i) {
                ll d = (1ll << (j - 1));
                sparse[i][j] = max(sparse[i][j - 1], sparse[min(n - 1, i
    + d)][j - 1]);
            }
        }
    }

    T query(ll l, ll r) {
        ll t = length[r - l];

        ll d = (1ll << t);

        return max(sparse[l][t], sparse[r - d][t]);
    }
};
```

4.14 Treap

```
struct Treap {
    Treap* left;
    Treap* right;
    ll x;
    unsigned int y;
    unsigned int size;
    ll dp;

    Treap(ll val = 0) {
        left = nullptr;
        right = nullptr;
        size = 1;
        x = val;
        y = mersenne();
        update(this);
    }

    friend ll get_dp(Treap* node) {
        if (node == nullptr)
            return 0;
        return node->dp;
    }

    friend unsigned int get_size(Treap* node) {
        if (node == nullptr)
            return 0;
        return node->size;
    }

    friend void update(Treap* node) {
        if (node == nullptr)
            return;
        node->size = 1 + get_size(node->left) + get_size(node->right);
        node->dp = max({node->x, get_dp(node->left), get_dp(node->right)
    });
    }

    // split by size
    friend pair<Treap*, Treap*> split_k(Treap* node, unsigned int k) {
        if (node == nullptr)
            return {nullptr, nullptr};
        if (get_size(node->left) < k) {
            auto res = split_k(node->right, k - get_size(node->left) - 1)
    ;
            node->right = res.first;
            update(node);
            return {node, res.second};
        } else {
            auto res = split_k(node->left, k);
```

```
            node->left = res.second;
            update(node);
            return {res.first, node};
        }
    }

    // split by dp: dp in left < k
    friend pair<Treap*, Treap*> split_dp(Treap* node, unsigned int k) {
        if (node == nullptr)
            return {nullptr, nullptr};
        if (get_dp(node) < k) {
            auto res = split_dp(node->right, k);
            node->right = res.first;
            update(node);
            return {node, res.second};
        } else {
            auto res = split_dp(node->left, k);
            node->left = res.second;
            update(node);
            return {res.first, node};
        }
    }

    friend Treap* merge(Treap* left, Treap* right) {
        if (left == nullptr)
            return right;
        if (right == nullptr)
            return left;
        if (left->y > right->y) {
            left->right = merge(left->right, right);
            update(left);
            return left;
        } else {
            right->left = merge(left, right->left);
            update(right);
            return right;
        }
    }

    friend Treap* insert(Treap* node, unsigned int pos, ll val) {
        auto res = split_k(node, pos);
        Treap* tr = new Treap(val);
        return merge(merge(res.first, tr), res.second);
    }

    friend Treap* erase(Treap* node, unsigned int pos) {
        auto res = split_k(node, pos + 1);
        auto left_res = split_k(res.first, pos);
        return merge(left_res.first, res.second);
    }

    // return {element, root}
    friend pair<Treap*, Treap*> kth_element(Treap* node, unsigned int k)
    {
        if (get_size(node) < k)
            return {nullptr, node};
        auto res = split_k(node, k + 1);
        auto left_res = split_k(res.first, k);
        return {left_res.second, merge(left_res.first, merge(left_res.
    second, res.second))};
    }

    // return sub segment [l, r) and remaineng Treap [0, l) + [r + 1,
    size)
    friend pair<Treap*, Treap*> cut(Treap* node, unsigned int l, unsigned
     int r) {
        auto res = split_k(node, r);
        auto left_res = split_k(res.first, l);
        return {left_res.second, merge(left_res.first, res.second)};
    }
};
```

5 Flows

5.1 Dinic

```
struct FlowSolver {
    struct DinicEdge {
        ll a, b, cap, flow;
    };

    vector<DinicEdge> edges;
    vector<vector<ll>> g;
    ll n;

    FlowSolver(ll N) {
        n = N;
```

```
        g.resize(n);
    }

    void add_edge(ll u, ll v, ll w) {
        g[u].push_back(edges.size());
        edges.push_back({u, v, w, 0});
        g[v].push_back(edges.size());
        edges.push_back({v, u, 0, 0});
    }

    bool bfs(ll s, ll t, vector<ll>& d) {
        queue<ll> q;
        d.assign(n, -1);
        d[s] = 0;
        q.push(s);

        while (!q.empty()) {
            ll v = q.front();
            q.pop();

            for (ll id : g[v]) {
                ll u = edges[id].b;
                if (d[u] == -1) {
                    if (edges[id].cap > edges[id].flow) {
                        d[u] = d[v] + 1;
                        q.push(u);
                    }
                }
            }
        }

        return d[t] != -1;
    }

    ll dfs(ll v, ll t, ll flow, vector<ll>& d, vector<ll>& ptr) {
        if (flow == 0) {
            return 0;
        }
        if (v == t) {
            return flow;
        }

        for (; ptr[v] < g[v].size(); ++ptr[v]) {
            ll id = g[v][ptr[v]];
            ll u = edges[id].b;

            if (d[u] == d[v] + 1) {
                ll add_flow = dfs(u, t, min(flow, edges[id].cap - edges[
    id].flow), d, ptr);
                if (add_flow > 0) {
                    edges[id].flow += add_flow;
                    edges[id ^ 1].flow -= add_flow;
                    return add_flow;
                }
            }
        }
        return 0;
    }

    ll dinic(ll s, ll t) {
        ll flow = 0;
        vector<ll> d;
        while (bfs(s, t, d)) {
            vector<ll> ptr(n, 0);
            while (ll add_flow = dfs(s, t, inf, d, ptr)) {
                flow += add_flow;
            }
        }
        return flow;
    }
};
```

6 Graphs

6.1 2 SAT

```
struct SATSolver {
    ll n;
    vector<vector<ll>> g, gt;
    vector<char> used;
    vector<ll> order, comp;

    void init(ll N) {
        n = N;
        g.assign((int)(2 * n), {});
        gt.assign((int)(2 * n), {});
    }
```

```
// literal: var in [0..n), val=false -> 2*var, val=true -> 2*var+1
ll lit(ll var, ll val) { return 2 * var + (val ? 1 : 0); }
ll neg(ll x) { return x ^ 1; }

void add_imp(ll a, ll b) {
    g[(int)a].push_back(b);
    gt[(int)b].push_back(a);
}

// (a_var is a_val) OR (b_var is b_val)
void add_or(ll a_var, ll a_val, ll b_var, ll b_val) {
    ll a = lit(a_var, a_val);
    ll b = lit(b_var, b_val);
    add_imp(neg(a), b);
    add_imp(neg(b), a);
}

void dfs1(ll v) {
    used[(int)v] = 1;
    for (ll to : g[(int)v]) if (!used[(int)to]) dfs1(to);
    order.push_back(v);
}

void dfs2(ll v, ll cl) {
    comp[(int)v] = cl;
    for (ll to : gt[(int)v]) if (comp[(int)to] == -1) dfs2(to, cl);
}

// returns empty vector if unsat; otherwise assignment[i] in {0,1}
vector<ll> solve() {
    used.assign((int)(2 * n), 0);
    order.clear();
    for (ll i = 0; i < 2 * n; ++i) if (!used[(int)i]) dfs1(i);

    comp.assign((int)(2 * n), -1);
    ll j = 0;
    for (ll i = 2 * n - 1; i >= 0; --i) {
        ll v = order[(int)i];
        if (comp[(int)v] == -1) dfs2(v, j++);
    }

    vector<ll> ans((int)n);
    for (ll i = 0; i < n; ++i) {
        if (comp[(int)(2 * i)] == comp[(int)(2 * i + 1)]) return {};
        ans[(int)i] = (comp[(int)(2 * i)] < comp[(int)(2 * i + 1)]);
    }
    return ans;
}
};
```

6.2 Euler Path

```
void cycle(ll prev, ll v, vector<vector<pair<ll, ll>>>& g, vector<ll>&
    ans) {
    while (!g[v].empty()) {
        auto iu = g[v].back();
        ll u = iu.second;
        g[v].pop_back();
        cycle(iu.first, u, g, ans);
    }
    if (prev >= 0)
        ans.push_back(prev);
}
```

6.3 Hopcroft Karp

```
struct HopcroftKarp {
    ll n1, n2;
    vector<vector<ll>> g;
    vector<ll> dist;
    vector<ll> mt1, mt2;

    void init(ll N1, ll N2) {
        n1 = N1;
        n2 = N2;
        g.assign((int)n1, {});
        mt1.assign((int)n1, -1);
        mt2.assign((int)n2, -1);
        dist.assign((int)n1, 0);
    }

    void add_edge(ll u, ll v) {
        // u in [0..n1), v in [0..n2)
        g[(int)u].push_back(v);
    }
```

```
    bool bfs() {
        queue<ll> q;
        for (ll i = 0; i < n1; ++i) {
            if (mt1[(int)i] == -1) {
                dist[(int)i] = 0;
                q.push(i);
            } else {
                dist[(int)i] = -1;
            }
        }

        bool found = false;
        while (!q.empty()) {
            ll v = q.front();
            q.pop();

            for (ll to : g[(int)v]) {
                ll u = mt2[(int)to];
                if (u == -1) {
                    found = true;
                } else if (dist[(int)u] == -1) {
                    dist[(int)u] = dist[(int)v] + 1;
                    q.push(u);
                }
            }
        }
        return found;
    }

    bool dfs(ll v) {
        for (ll to : g[(int)v]) {
            ll u = mt2[(int)to];
            if (u == -1 || (dist[(int)u] == dist[(int)v] + 1 && dfs(u)))
    {
                mt1[(int)v] = to;
                mt2[(int)to] = v;
                return true;
            }
        }
        dist[(int)v] = -1;
        return false;
    }

    ll solve() {
        ll matching = 0;
        while (bfs()) {
            for (ll i = 0; i < n1; ++i) {
                if (mt1[(int)i] == -1) {
                    if (dfs(i)) {
                        ++matching;
                    }
                }
            }
        }
        return matching;
    }
};
```

6.4 Hungarian

```
// a size is (n + 1) x (m + 1), 1-indexing
// n <= m; if n < m than a[i][j] must be >= 0
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& a, int n, int m
    ) {
    vector<ll> u(n + 1, 0), v(m + 1, 0);
    vector<int> p(m + 1, 0), way(m + 1, 0);
    for (int i = 1; i <= n; ++i) {
        int j0 = 0;
        p[j0] = i;

        vector<char> used(m + 1, 0);
        vector<ll> minv(m + 1, inf);
        do {
            used[j0] = true;
            int i0 = p[j0];
            int j1 = -1, delta = inf;

            // run step of dfs from i0
            for (int j = 1; j <= m; ++j) {
                if (used[j]) {
                    continue;
                }

                int cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) {
                    minv[j] = cur;
                    way[j] = j0;
                }
```

```
                if (minv[j] < delta) {
                    delta = minv[j];
                    j1 = j;
                }
            }

            // update potentials
            for (int j = 0; j <= m; ++j) {
                if (used[j]) {
                    u[p[j]] += delta;
                    v[j] -= delta;
                } else {
                    minv[j] -= delta;
                }
            }

            j0 = j1;
        } while (p[j0] != 0);

        while (j0) {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        }
    }

    return {-v[0], p};
}
```

6.5 Initial Pairing

```
int init_pairing(int n1, int n2, const vector<vector<int>>& g, vector<int
    >& mt, vector<int>& rmt) {
    auto PACK = [](int x, int y) { return ((ll)x << 30) + y; };
    auto SUNPACK = [](int x) {
        const static ll mask = (1ll << 30) - 1;
        return x & mask;
    };

    vector<int> deg(n1, 0);
    vector<vector<int>> ng(n1);
    vector<vector<pair<int, int>>> rg(n2);
    vector<int> next(n1, 0);
    set<ll> que;
    for (int i = 0; i < n1; ++i) {
        for (auto j : g[i]) {
            ++deg[i];
            rg[j].push_back({i, ng[i].size()});
            ng[i].push_back(j);
        }
        que.insert(PACK(deg[i], i));
    }

    int pairing = 0;

    while (!que.empty()) {
        auto res = *que.begin();
        que.erase(res);

        int i = SUNPACK(res);
        if (deg[i] == 0) {
            continue;
        }

        while (ng[i][next[i]] == -1) {
            ++next[i];
        }
        int j = ng[i][next[i]++];

        deg[i] = 0;
        for (auto ks : rg[j]) {
            int k = ks.first;
            int sz = ks.second;
            if (deg[k] != 0) {
                ng[k][sz] = -1;
                que.erase(PACK(deg[k], k));
                --deg[k];
                que.insert(PACK(deg[k], k));
            }
        }
        mt[j] = i;
        rmt[i] = j;
        ++pairing;
    }
    return pairing;
}

vector<int> fast_kuhn(int n1, int n2, vector<vector<int>>& g) {
```

```
    vector<char> used(n1, 0);
    vector<int> mt(n2, -1), rmt(n1, -1);

    init_pairing(n1, n2, g, mt, rmt);

    int mark = 0;

    for (int run = 1; run;) {
        ++mark;
        run = 0;
        for (int i = 0; i < n1; ++i) {
            if (rmt[i] == -1 && kuhn_dfs(i, used, mark, g, mt)) {
                run = 1;
            }
        }

        for (int j = 0; j < n2; ++j) {
            if (mt[j] != -1) {
                rmt[mt[j]] = j;
            }
        }
    }

    return mt;
}
```

6.6 Kuhn

```
bool kuhn_dfs(int v, vector<char>& used, char mark, vector<vector<int>>&
    g, vector<int>& mt) {
    if (used[v] == mark) {
        return false;
    }
    used[v] = mark;

    for (int u : g[v]) {
        if (mt[u] == -1 || kuhn_dfs(mt[u], used, mark, g, mt)) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

vector<int> kuhn(int n1, int n2, vector<vector<int>>& g) {
    vector<char> used(n1, 0);
    vector<int> mt(n2, -1);

    int mark = 1;

    for (int i = 0; i < n1; ++i) {
        if (kuhn_dfs(i, used, mark, g, mt)) {
            ++mark;
        }
    }

    return mt;
}
```

7 Number Theory

7.1 Eratosthenes

```
void erat(ll N, vector<ll>& primes, vector<ll>& mi) {
    mi.assign(N, 0);
    for (int i = 2; i < N; ++i) {
        if (mi[i] == 0) {
            primes.push_back(i);
            mi[i] = i;
        }

        for (int j = 0; j < primes.size() && primes[j] <= mi[i] && i *
    primes[j] < N; ++j) {
            mi[primes[j] * i] = primes[j];
        }
    }
}
```

7.2 Miller Rabin

```
bool _miller_rabin_test(ll n, ll d, ll s, ll a) {
    ll x = bin_pow_mod(a, d, n);
    for (int i = 0; i < s; ++i) {
        ll y = ((__int128_t)x * x) % n;
        if (y == 1 && x != 1) {
            return false;
```

```
        }

        x = y;
    }
    if (x != 1) {
        return false;
    }
    return true;
}

bool is_prime_ml(ll n) {
    if (n <= 1) {
        return false;
    }
    // all primes
    vector<ll> as = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    ll d = n - 1;
    ll s = 0;
    while ((d & 1) == 0) {
        ++s;
        d >>= 1;
    }
    for (ll a : as) {
        if (n == a) {
            return true;
        }
        if (n % a == 0) {
            return false;
        }
        if (!_miller_rabin_test(n, d, s, a)) {
            return false;
        }
    }
    return true;
}
```

8 Strings

8.1 LCP

```
// p - suffix massive output
// c - suffix massive classes
vector<ll> calc_lcp(ll n, const string& s, const vector<ll>& c, const
    vector<ll>& p) {
    vector<ll> lcp(n - 1, 0);

    ll pref = 0;

    for (int i = 0; i < n - 1; ++i) {
        ll pos = c[i];
        ll j = p[pos - 1];
        // j .. i

        while (i + pref < n && j + pref < n && s[i + pref] == s[j + pref
    ]) {
            ++pref;
        }
        lcp[pos - 1] = pref;
        pref = max(0ll, pref - 1);
    }

    return lcp;
}
```

8.2 Prefix Function

```
vector<ll> prefix_function(string s) {
    ll n = s.size();
    vector<ll> p(n, 0);
    for (int i = 1; i < n; ++i) {
        ll k = p[i - 1];
        while (k > 0 && s[i] != s[k])
            k = p[k - 1];
        if (s[i] == s[k])
            ++k;
        p[i] = k;
    }
    return p;
}
```

8.3 Suffix Massive

```
void radix_sort(vector<pair<pair<ll, ll>, ll>>& prs) {
    ll n = prs.size();
    vector<ll> cnt(n, 0);
    for (auto mem : prs) {
```

```
        cnt[mem.first.first]++;
    }
    vector<ll> ptr(n, 0);
    for (int i = 1; i < n; ++i) {
        ptr[i] = ptr[i - 1] + cnt[i - 1];
    }

    vector<pair<pair<ll, ll>, ll>> new_prs(n);

    for (auto mem : prs) {
        ll i = mem.first.first;

        new_prs[ptr[i]++] = mem;
    }
    prs = new_prs;
}

vector<ll> suffix_array(string s) {
    s += '$';

    ll n = s.size();

    vector<ll> p(n), c(n);

    {  // k == 0
        vector<pair<ll, ll>> prs(n);
        for (int i = 0; i < n; ++i) {
            prs[i] = {s[i], i};
        }
        sort(all(prs));
        for (int i = 0; i < n; ++i) {
            p[i] = prs[i].second;
        }

        c[p[0]] = 0;
        for (int i = 1; i < n; ++i) {
            if (prs[i].first == prs[i - 1].first) {
                c[p[i]] = c[p[i - 1]];
            } else {
                c[p[i]] = c[p[i - 1]] + 1;
            }
        }
    }

    ll k = 0;

    while ((1ll << k) < n) {
        vector<pair<pair<ll, ll>, ll>> prs(n);
        for (int i = 0; i < n; ++i) {
            ll t = p[i];
            ll j = ((t - (1ll << k)) % n + n) % n;
            prs[i] = {{c[j], c[t]}, j};
        }
        radix_sort(prs);

        for (int i = 0; i < n; ++i) {
            p[i] = prs[i].second;
        }

        c[p[0]] = 0;
        for (int i = 1; i < n; ++i) {
            if (prs[i].first == prs[i - 1].first) {
                c[p[i]] = c[p[i - 1]];
            } else {
                c[p[i]] = c[p[i - 1]] + 1;
            }
        }

        ++k;
    }

    vector<ll> ans;
    ans.reserve((int)n - 1);
    for (int i = 1; i < n; ++i)
        ans.push_back(p[i]);
    return ans;
}
```

8.4 Z Function

```
vector<ll> z_function(string s) {
    ll n = s.size();
    vector<ll> z(n, 0);
    ll r = 0;
    for (int i = 1; i < n; ++i) {
        if (z[i - r] < r + z[r] - i)
            z[i] = z[i - r];
        else {
```

```
        z[i] = max(r + z[r] - i, 0ll);
        while (i + z[i] < n && s[i + z[i]] == s[z[i]])
            ++z[i];
        r = i;
            }
        }
        return z;
    }
```